Facade Pattern for a Home Theater

Assignment 4 | Software Design Patterns | ShP-2216
Astana IT University - School of Computer Engineering

Student Olzhabekov Ali **Group** SE-2515

1 Introduction

This project implements the Home Theater option from the assignment. Starting a movie involves five components: Lights, Screen, DvdPlayer, Amplifier and Projector. HomeTheaterFacade exposes watchMovie(title), endMovie() and status(), so the normal client can run complete sessions without configuring individual devices.

The subsystem has ordering constraints. A player must have a movie before playback; an amplifier requires an active source; a projector requires both playback and a lowered screen. A playing movie cannot be replaced or ejected. These rules belong to the subsystem classes. The facade supplies the sequence that satisfies them and coordinates cleanup when a step fails.

Component	Responsibility
DvdPlayer	Validate and normalize titles; manage loading, playback, stopping and ejecting.
Amplifier	Validate volume and connect an actively playing audio source.
Projector	Check video playback and screen position; maintain the projection connections.
Screen	Maintain the raised or lowered position required by projection.
Lights	Validate brightness; allow the facade to restore the pre-session level.

Why a facade is useful

Without a facade, every ordinary client would construct the five objects, establish their links, remember the setup and shutdown order, and handle partial failure. The facade turns this recurring workflow into a small API. Main uses only that API; it neither constructs nor calls subsystem objects. Subsystems do not reference HomeTheaterFacade and remain publicly available for advanced clients.

The implementation is a deterministic in-memory device model. It does not control physical equipment or decode media files. Its non-trivial behavior is expressed by state transitions, cross-component preconditions and failure recovery, rather than messages that merely claim devices were switched on.

2 UML class diagram

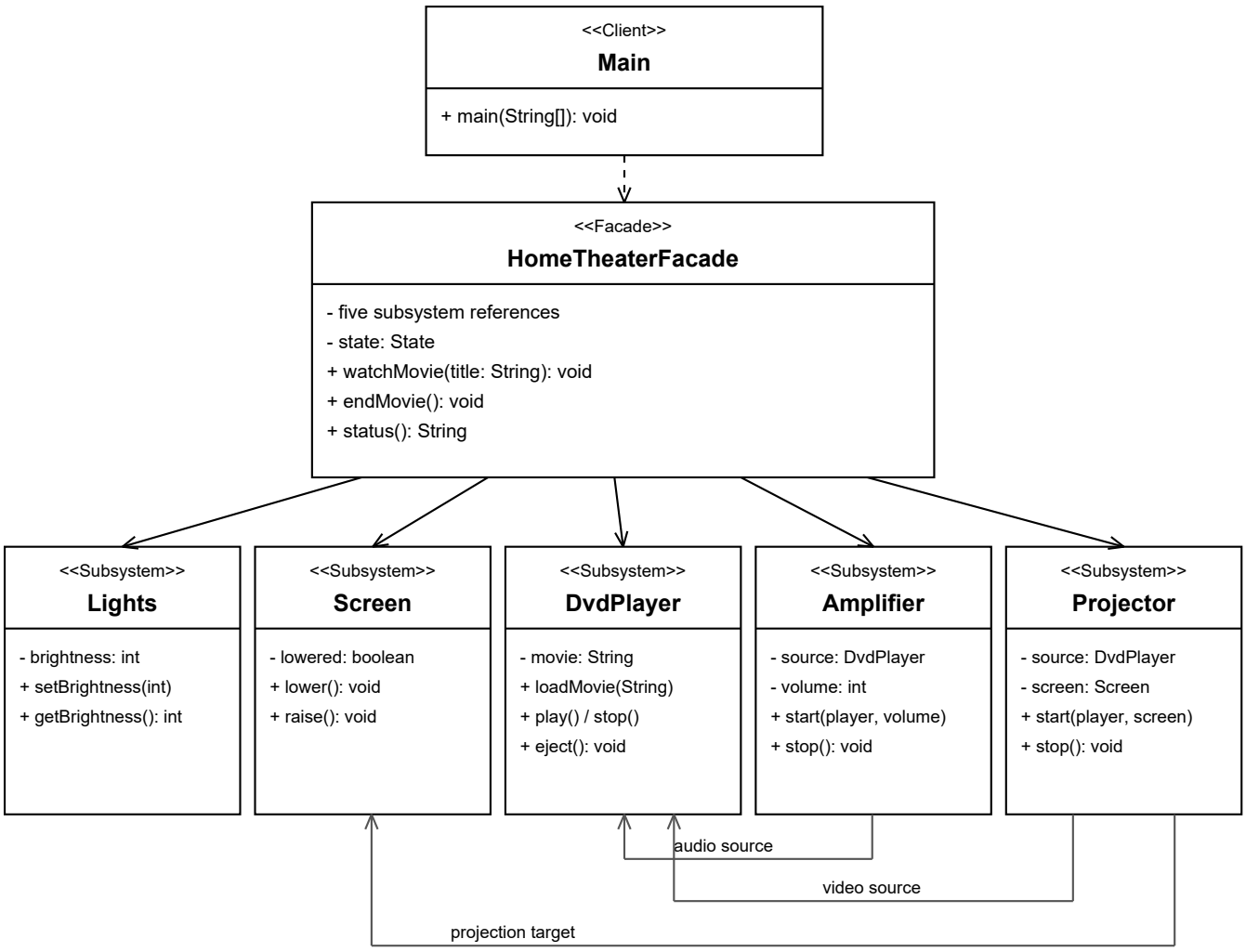


Figure 1. Main depends on the facade. The facade holds five subsystem references. Amplifier and Projector use DvdPlayer directly; Projector also uses Screen. These references point into the subsystem, never back to the facade.

The diagram abbreviates method parameters and omits some accessors and private coordination fields for readability. The complete editable PlantUML diagram is included as docs/facade.puml. Solid arrows show navigable references, and the dashed arrow shows the client dependency. No inheritance relationship is implied between the facade and its subsystems.

The normal client

```
HomeTheaterFacade theater = new HomeTheaterFacade();
theater.watchMovie("Interstellar");
System.out.println(theater.status());
theater.endMovie();
```

The facade initializes the default devices internally. Its public constructor takes no subsystem arguments, and its public methods exchange only a movie title or a readable status. A package-private constructor supports collaborator injection in tests without widening the public client API.

3 Clean Code principles

The annotated excerpts below correspond to the submitted implementation. Each principle addresses a concrete source of complexity in the movie workflow.

1 Meaningful names for the simplified API

```
// Client calls express movie-session goals
theater.watchMovie(title);
System.out.println("After watchMovie: " + theater.status());
theater.endMovie();
```

The method names express user goals. A caller does not have to infer that several technical operations together mean starting a movie. The client can read and write session code in terms of watching, status and ending. Names such as previousBrightness and currentMovie make the saved coordination state explicit.

2 Single responsibility for coordination

```
// Excerpt from watchMovie: orchestration, not device logic
player.loadMovie(movie);
lights.setBrightness(MOVIE_BRIGHTNESS);
screen.lower();
player.play();
amplifier.start(player, MOVIE_VOLUME);
projector.start(player, screen);
```

The facade selects the order and supplies defaults. It delegates title validation, playback rules, volume validation and projection checks to their owning classes. A change in the player loading rule belongs in DvdPlayer; the facade continues to call loadMovie. This avoids duplicating subsystem logic in a large central class.

3 Hide subsystem details from ordinary clients

```
// Main: no subsystem construction or direct device calls
HomeTheaterFacade theater = new HomeTheaterFacade();
theater.watchMovie(title);
```

Private subsystem fields keep device references out of Main. The facade does not return a Projector, require a DvdPlayer parameter or expose a device configuration structure. Advanced use remains possible by creating the public subsystem classes directly, which is verified by a separate direct-use test.

3 Clean Code principles continued

4 Small subsystem classes enforce their own rules

```
// Projector.start validates its collaboration requirements
if (!source.isPlaying()) {
    throw new IllegalStateException("Video source must be playing");
}
if (!screen.isLowered()) {
    throw new IllegalStateException(
            "Lower the screen before starting projection");
}
```

Projector knows the rules for projection and stores its source and target. Amplifier manages audio and volume; DvdPlayer manages movie playback. Lights and Screen model the remaining room state. These focused classes are useful without any facade and make an incorrect setup order observable as an error.

5 Handle coordination failures explicitly

```
// watchMovie preserves startup and cleanup failure information
} catch (RuntimeException failure) {
    try {
        restoreRoom();
    } catch (RuntimeException cleanupFailure) {
        failure.addSuppressed(cleanupFailure);
    }
    throw new TheaterOperationException(
            "Could not start movie: " + movie, failure);
}
```

The session becomes WATCHING only after every setup step succeeds. A startup failure triggers cleanup. If cleanup also fails, the cause and suppressed information are retained, status remains Cleanup required, and new sessions are blocked. endMovie can retry cleanup. The code never reports Ready after an incomplete cleanup.

6 Reuse the shutdown policy instead of duplicating it

```
// restoreRoom defines one ordered list for normal and failed sessions
List<Runnable> steps = List.of(projector::stop, amplifier::stop,
        player::stop, player::eject, screen::raise,
        () -> lights.setBrightness(previousBrightness));
```

Both endMovie and failed startup call restoreRoom. Its loop attempts every cleanup step and accumulates failures. The final Ready state is assigned only after success. Sharing this policy prevents the rollback path from forgetting a device or restoring lights differently from normal shutdown.

4 Verification and conclusion

Behavioral verification

All 16 tests and the demonstration passed on Temurin Java 17 in both Windows and Linux GitHub Actions jobs. Local Windows execution also passed on Java 17 and Oracle JDK 25.0.1. Compilation uses --release 17 -encoding UTF-8 -Xlint:all -Werror; class-file version 61 was confirmed. Detailed results are recorded in docs/verification.txt.

Verified Java 17 workflow run 39141879643 - both operating-system jobs completed successfully.

Area	Verified behavior
Subsystem rules	Invalid brightness, volume and titles; missing movie; playback before projection; screen position; stopping before ejecting.
Complete sessions	One facade call starts all devices; shutdown stops them, ejects the movie and restores original brightness.
Client behavior	Repeat start is rejected; idle shutdown is safe; multiple sessions work; direct subsystem use remains available.
Failure handling	Startup rollback, preserved cause, continued cleanup after failure, blocked restart and a successful later cleanup retry.

Main demonstrates a complete session, a rejected second start, repeated shutdown and a new movie. It also accepts a custom movie title. Invalid command-line input exits with code 2. Public facade operations expose no subsystem types; subsystem source files contain no HomeTheaterFacade dependency.

What the facade hides and what stays accessible

The facade hides device creation, source connections, setup and shutdown order, standard volume and brightness, and cleanup coordination. Ordinary clients depend on the movie-session API. Public subsystem classes remain directly accessible for clients that need custom device control. A direct-use test constructs and operates all five components without a facade.

Trade-offs and limits

The facade reduces repeated client orchestration but depends on five concrete classes. Adding unrelated operations could make it too large, so this implementation stays focused on movie sessions. Fixed settings keep the API simple while advanced control requires direct subsystem use. Cleanup uses compensating operations, which may themselves fail; it is not an atomic transaction. One instance represents one room and assumes single-threaded access. The model simulates device state rather than actual media playback.

GitHub repository

https://github.com/maxiszero/assignment-4-facade-pattern

References

Assignment 4 Facade Pattern Instructions, supplied course handout.
Refactoring.Guru - Facade (assigned conceptual reading).
Oracle Java SE 17 - Throwable (causes and suppressed exceptions).